

Implementation Details of the Reconstructed Explicit MPC

sumo_cra_traffic_sim

September 10, 2026

Abstract

This report documents the explicit-MPC controller currently implemented in `scripts/_controller.py` and its integration into `scripts/run_sim.py`. The controller is a reduced, paper-aligned reconstruction of the Eco-MPC quadratic program. It uses a fixed-horizon linear vehicle model, converts the online QP into affine active-set laws, and evaluates those laws through a region lookup at runtime. An OSQP solve remains available as a fallback when no stored region is valid. The report also describes the distinction between connected preview and unconnected constant-acceleration preview, safety filtering, region caching, and the current full-SUMO-step runtime measurement.

1 Scope and interpretation

The implementation is intended for an interpretable explicit-MPC experiment, not as a binary replacement for the original compiled controller. The online MPC used by the project is provided through the compiled `libpcc.so` path; its complete QP construction, reference packing, and solver details are not available in the Python workspace. Therefore, the explicit controller reconstructs the main paper formulation and uses the same reconstructed QP for its online validation and offline region generation.

The current implementation preserves the original online MPC path. The explicit modes are selected independently with:

```
python3 scripts/run_sim.py --num_sv 5 --no_gui Hwy ExplicitUnconnected
python3 scripts/run_sim.py --num_sv 5 --no_gui Hwy ExplicitConnected
python3 scripts/run_sim.py --num_sv 5 --no_gui Hwy MPC
```

The explicit implementation is therefore best interpreted as a benchmark for the reconstructed QP and as a study of explicit-MPC computation, rather than as a guaranteed exact surrogate for the unavailable compiled QP.

2 Configuration used by the simulation

The explicit controller is configured in `run_sim.py` as follows. The numerical values below are the values passed by the current simulation entry point; the configuration class also has defaults for standalone use.

The value of dt is taken from the explicit-MPC configuration and is not automatically changed to match SUMO's internal integration step. The horizon and weights are part of the region-cache signature, so changing them selects a different cache file.

Parameter	Current simulation value
Sampling time	$dt = 0.5$ s
Prediction/control horizon	$N = 20$
Region-discovery samples	10,000
Acceleration bounds	$-3 \leq u \leq 3$ m/s ²
Velocity bounds	$0 \leq v \leq 35$ m/s
Vehicle length	7 m
Gap and time-headway defaults	0
Gap weight	$q_{\text{gap}} = 1$
Acceleration weight	$q_{\text{acc}} = 2000$
Command weight	$q_{\text{command}} = 2000$
Slack weight	$q_{\text{slack}} = 10^6$
Acceleration-speed constraints	disabled
CBF safety filter	enabled unless explicitly disabled

Table 1: Explicit-MPC configuration used by the current simulation entry point.

3 Prediction model

The state is

$$x_k = \begin{bmatrix} s_k & v_k & a_k \end{bmatrix}^T,$$

where s , v , and a are position, speed, and acceleration. The model in the Python implementation is

$$x_{k+1} = Ax_k + Bu_k,$$

with

$$A = \begin{bmatrix} 1 & dt & \frac{1}{4}dt^2 \\ 0 & 1 & \frac{1}{2}dt \\ 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{4}dt^2 \\ \frac{1}{2}dt \\ 1 \end{bmatrix}.$$

The prediction matrices are condensed offline. For every stage j , the implementation stores matrices such that

$$x_j = X_j\theta + U_jU,$$

where $U = [u_0, \dots, u_{N-1}]^T$ and θ contains the current vehicle states and, for the preview formulation, future preceding-vehicle positions. This makes both the objective and constraints affine or quadratic functions of the parameter vector.

4 Parameterization of connected and unconnected cases

The current unconnected controller is a wrapper around the same preview-parameterized controller used for the connected case. The distinction is the source of the preview, not the QP structure.

4.1 Connected preview

The connected input is

$$\theta_c = \begin{bmatrix} s_e + L & v_e & a_e & s_p & v_p & a_p & s_{p,1}^{\text{preview}} & \dots & s_{p,N}^{\text{preview}} \end{bmatrix}^T.$$

Here L is the vehicle length. The ego-position shift is applied before the QP is formed so that the reconstructed gap convention is consistent with the vehicle geometry. The QP uses the communicated

preceding-vehicle position preview as its reference trajectory. A velocity preview is accepted by the interface for compatibility and diagnostics, but the current objective uses the position preview.

4.2 Unconnected preview

The unconnected controller receives only the current preceding-vehicle state and estimates future positions under constant acceleration. At each preview stage it advances the predicted speed using the current preceding-vehicle acceleration, clips speed to the configured velocity limits, and advances the position using the clipped speed:

$$\hat{v}_{p,j} = \text{clip}(\hat{v}_{p,j-1} + dt a_p, v_{\min}, v_{\max}),$$

$$\hat{s}_{p,j} = \hat{s}_{p,j-1} + dt \hat{v}_{p,j}.$$

The resulting N positions are passed into the same connected preview QP. Consequently, both modes use the same parameter dimension, region set, and runtime affine-law evaluation. Only the preview source differs.

5 Reduced quadratic program

The decision vector contains the future accelerations and three slack vectors:

$$z = \begin{bmatrix} U \\ \epsilon_v^+ \\ \epsilon_v^- \\ \epsilon_g \end{bmatrix}, \quad U \in \mathbb{R}^N,$$

with one slack variable per stage for upper velocity, lower velocity, and minimum-gap constraints. The QP is assembled in the standard parametric form

$$\min_z \frac{1}{2} z^\top H z + q(\theta)^\top z, \quad G z \leq w + W \theta.$$

For each prediction stage, the objective includes:

1. quadratic predicted-gap tracking;
2. quadratic predicted-acceleration penalty;
3. quadratic command penalty on each u_j ;
4. a large linear penalty on each slack variable.

In compact notation, the implemented objective has the form

$$J = \sum_{j=1}^N \left(q_{\text{gap}} \|g_j + g_{\text{desired}}\|^2 + q_{\text{acc}} a_j^2 + q_{\text{command}} u_{j-1}^2 \right) + q_{\text{slack}} \mathbf{1}^\top \epsilon.$$

The exact affine sign convention for the desired gap follows the assembled matrices in `_build_qp`; the implementation's primary gap expression is $C_{\text{gap}} x_j - s_{p,j}$ with

$$C_{\text{gap}} = \begin{bmatrix} 1 & h & 0 \end{bmatrix},$$

where h is the configured time headway.

6 Constraints

The fixed acceleration bounds are hard constraints:

$$u_{\min} \leq u_j \leq u_{\max}.$$

The predicted speed bounds are softened with nonnegative slacks:

$$v_j \leq v_{\max} + \epsilon_{v,j}^+, \quad v_j \geq v_{\min} - \epsilon_{v,j}^-.$$

The minimum-gap constraint is also softened:

$$s_{p,j} - s_j \geq g_{\min} - \epsilon_{g,j}.$$

All three slack families are constrained to be nonnegative. The large slack penalty makes violations expensive while keeping the QP solvable for states that are temporarily infeasible.

The paper’s acceleration-versus-speed envelope can be assembled by setting `include_acceleration_speed_constraints=True`. It is currently disabled in the simulation configuration, as requested. The maximum and minimum acceleration constraints remain active independently of this switch.

7 Offline active-set region construction

For a fixed active constraint set $\mathcal{A}$, the KKT equations are solved offline. Since $q(\theta)$ and the constraint right-hand side are affine in θ , the solution is affine within that active set:

$$z(\theta) = z_0 + Z\theta.$$

The first row gives the explicit control law:

$$u_0(\theta) = K_{\mathcal{A}}\theta + k_{\mathcal{A}}.$$

The current implementation discovers candidate active sets by sampling feasible-looking state and preview parameters. For every sample it:

1. solves the reconstructed QP with OSQP;
2. identifies constraints close to their upper bound;
3. keeps each previously unseen active set;
4. solves its KKT linear system;
5. stores the affine primal and dual expressions.

This is a sampled active-set discovery process. It does not claim to enumerate every possible region of the full parameter space. Consequently, increasing `sample_count` can improve observed coverage but increases offline OSQP work and may still not guarantee a complete partition.

At runtime, the controller checks the stored KKT inequalities for each region. For a valid region it evaluates the affine law using NumPy, without solving a QP. The stored primal and dual inequalities are checked directly rather than using only a loose axis-aligned parameter box.

8 OSQP fallback and safety filter

If no stored region satisfies the KKT validity tests, the controller solves the same reconstructed QP online with OSQP. The fallback count and region-hit count are retained as diagnostics. This makes the controller operational outside the sampled region set, but introduces runtime spikes and means that the measured runtime depends on region coverage.

After either affine evaluation or OSQP fallback, the existing control-barrier function safety limit is applied when CBF is enabled:

$$a_{\text{CBF}}^{\text{max}} = \frac{v_p - v_e + \alpha(s_p - s_e - L - \tau v_e)}{\tau}.$$

The command is then limited by this safety bound and clipped to the configured acceleration interval. CBF activation can be selected from the command line:

```
python3 scripts/run_sim.py --disable_explicit_cbf Hwy ExplicitConnected
```

Without the flag, explicit-MPC CBF filtering is enabled. Diagnostics report whether a region was used, whether OSQP fallback occurred, the raw affine/QP acceleration, the CBF limit, and whether CBF overrode the raw command.

9 Region caching

Generated regions are saved under:

```
scripts/.explicit_mpc_cache/
```

The cache filename is configuration-specific. Its hash includes the mode, sampling time, horizon, vehicle geometry, bounds, objective weights, slack weight, sample count, random seed, optional acceleration-speed switch, and parameter dimension. On later runs, a matching cache is loaded instead of re-running region discovery. Cache files are written through a temporary file and atomic replacement. The cache directory is ignored by Git because it is a generated artifact.

The cache avoids repeated offline construction, but it does not remove online lookup cost or OSQP fallback cost. Changing any QP-defining parameter creates a new cache key.

10 SUMO integration and runtime measurement

The explicit controller is evaluated for each follower vehicle; the leading vehicle follows the prescribed driving-cycle speed. A single explicit region set is shared by all followers in a simulation. The current runtime timer in `run_sim.py` uses `time.perf_counter()` and is defined as:

1. start immediately before `simulationStepForward()`;
2. include SUMO advancement, state collection, every follower controller evaluation, control assignment, preview updates, and per-step statistics;
3. stop before optional simulation CSV logging and the deliberate `sleep(0.001)` pacing delay.

Thus, the recorded value is one complete SUMO/control-step runtime, not the runtime of a single vehicle call. It is appended to `Runtime_<controller>.csv` in milliseconds. Each row contains:

```
vehicle_count, mean_ms, minimum_ms, maximum_ms, standard_deviation_ms
```

The summary currently removes samples equal to the maximum runtime before calculating the reported statistics, retaining the existing warm-up/outlier handling. For rigorous reporting, the raw per-step values should also be retained when comparing controllers, especially because OSQP fallbacks and operating-system scheduling can create occasional spikes.

11 Important limitations for interpretation

- The explicit law is exact for a stored active set of the reconstructed QP, subject to numerical tolerances in the KKT checks.
- It is not guaranteed to reproduce the compiled online MPC because the compiled controller's complete formulation and input packing are unavailable.
- More discovery samples can increase region coverage, but they increase offline CPU and memory use and do not correct a mismatch in the reconstructed QP.
- A high-dimensional connected preview produces a more difficult region space. The current unconnected mode uses the same preview dimension by generating its preview locally, so its computational comparison isolates preview source more than parameter dimension.
- CBF filtering is applied after the explicit law. Therefore, the final command can be piecewise affine only before the CBF clamp; the complete closed-loop mapping includes the safety saturation.

12 Source locations

The principal implementation locations are:

- `scripts/_controller.py`: model, QP assembly, active-set region generation, cache loading/saving, affine evaluation, fallback, and CBF;
- `scripts/run_sim.py`: controller selection, connected and unconnected inputs, SUMO integration, diagnostics, and full-step runtime;
- `runtime_analysis.sh`: headless runtime-analysis command loop.